A Privacy-Preserving, Censorship-Resistant, Decentralized Order Book

I. Introduction

Decentralized exchanges (DEXs) have grown into one of the most heavily used classes of blockchain applications, and limit order books have become their dominant trading architecture. A DEX processes an order in the same three stages as a traditional centralized exchange (CEX). In *placement*, a trader submits a buy or sell order specifying the trading pair, the side, the limit price, and the order quantity. In *matching*, the order book pairs crossing orders under a deterministic priority rule, typically price priority followed by time priority or some protocol-specific tie-breaking policy. In *settlement*, the executed trade is cleared: the corresponding assets are transferred between the two parties, their balances are updated, and the affected orders are amended or removed from the book. What separates a DEX from a CEX is that no trusted third party acts as trading intermediary or asset custodian [1]: the order book and every transition applied to it are maintained by the ledger and re-validated by every node. This is what gives a DEX its two defining advantages over a CEX—transparency, since the state of the market is public and independently verifiable rather than asserted by an operator, and censorship resistance, since no single entity can selectively refuse, reorder, or withhold a trader's order.

However, the very openness that yields these advantages also creates a privacy problem that a CEX does not have. On a public order book, the quantity of every resting order and the balance of every account are published in plaintext, and quantity is precisely the signal an adversary needs. Exposing a large order reveals both its sustained trading intent and its potential price impact, and a *parasitic trader* can act on that disclosure: it accumulates a position in the same direction, taking the liquidity the large trader would otherwise have obtained, and then supplies that liquidity back at worse prices, raising the execution cost borne by the large order [2], [3]. The attack therefore has a necessary first step—*target selection*—in which the adversary must find a large order and tell it apart from the ordinary orders around it, since positioning is costly and only repays itself above a size threshold. Public on-chain order books such as dYdX and Injective publish exactly the field that makes this step trivial, and they publish it under conditions strictly worse than a CEX: the book and its entire history are permanently readable by anyone at no cost, replayable off-line, and accompanied by account balances that expose each trader's latent capacity to trade even before an order is placed.

Existing privacy-preserving exchange designs fall into three classes, each of which surrenders something essential. Systems built around an *off-chain or centralized matcher* keep order details away from the public, but reintroduce a trusted entity into the trading workflow: the matcher learns the sensitive order information it is supposed to protect, and its discretion over what to include weakens censorship resistance [4], [5]. *Fully hiding* designs conceal balances, sides, prices, and quantities alike, and thereby operate as dark pools; but withholding pre-trade information removes price-relevant signals from the public market, and empirical evidence shows that a high level of dark trading degrades price discovery and raises adverse-selection risk in the lit market [6]. *Iceberg orders* occupy the middle ground by displaying only a peak and replenishing it as it executes, but the concealment is heuristic rather than cryptographic: both the presence and the magnitude of hidden liquidity are predictable from observable order-book dynamics [7], [8], and detection algorithms recover an iceberg's hidden volume from the replenishment pattern its own execution leaves behind [9], [10]. It therefore remains an open problem to protect the privacy of order quantities and balances while preserving both the decentralization of the venue and the public price discovery a lit order book exists to provide.

This paper answers that question with Invisibook, and the answer begins from a deliberate separation of what an order book publishes from what it must conceal. To preserve price discovery, we publish every order's trading pair, side, limit price, and fee in plaintext, so that matching remains an ordinary deterministic priority rule executed and verified by the chain in public, exactly as in a fully transparent DEX. To protect the trader, we publish no quantity at all: an order carries its quantity only as a hiding, binding commitment, balances are held in a shielded representation, and every quantity-dependent step is deferred to a maliciously secure two-party computation run *by the two matched traders themselves*, whose result the chain verifies through a collaborative zero-knowledge proof [11] anchored to the on-chain commitments. Price discovery therefore proceeds on public data, while target selection is denied the one input it consumes. This choice raises two technical challenges. *First*, a chain that sees quantities only as commitments cannot compare, split, or sum them, and so

Network and Distributed System Security (NDSS) Symposium 2026
23 - 27 February 2026 , San Diego, CA, USA
ISBN 979-8-9919276-8-0
https://dx.doi.org/10.14722/ndss.2026.[23|24]xxxx
www.ndss-symposium.org

cannot walk the book to fill an incoming order or even detect when it has been exhausted; matching must be redefined so that it stays deterministic and publicly verifiable under this restriction, and so that a partially filled order can return to the book with its residual still concealed. *Second*, with no server and no trusted third party, the party each trader must be protected against is its own co-computing counterparty: a single corrupted party out of two, i.e. a dishonest majority with arbitrary deviation. The protocol must bind the computation's inputs to the quantities previously committed on chain, prevent either party from unilaterally corrupting the published result while learning the true one, and contend with stalling—a matched pair is locked and can no longer be cancelled, so simply withholding a message freezes the counterparty's order and capital, and no cryptographic check detects silence.

We summarize our contributions as follows.

- **A decoupled DEX architecture.** We propose the first order-book DEX design that decouples public price discovery from order-size privacy, publishing side and limit price while hiding quantity cryptographically rather than by display policy. Because the guarantee rests on the hiding property of a commitment scheme rather than on the empirical difficulty of inverting an execution pattern, the adversary's target-selection step is neutralized, and no strategy conditioned on inferred order size can outperform guessing.
- **A settlement protocol secure in the dishonest-majority setting.** We design an off-chain settlement protocol between the two matched traders that combines SPDZ-style maliciously secure two-party computation [12] with collaborative zero-knowledge proofs, together with a two-step on-chain assembly that verifies the proof before any value is opened and thereby denies a corrupted party the ability to spoil the public result while recovering the true one privately. An on-chain challenge-and-freeze mechanism further makes stalling unprofitable by imposing an opportunity cost on the delaying party while releasing the compliant party's order.
- **Implementation and evaluation.** [XinRan: wait for section Implementation]

II. Navigating the Privacy–Discovery Dilemma in Decentralized Exchanges

An order book must publish enough to let the market form a price, and must publish little enough to keep its traders from being preyed upon. This section develops that dilemma: how an order book works and why exposure is costly (§II-A), why the countermeasures the industry already has fail—and fail worse on chain (§II-B)—and what design point a privacy-preserving DEX should therefore aim at (§II-C).

A. Order Books and the Anatomy of Parasitic Trading

a) Limit order books.: A limit order book maintains the active buy and sell orders for a trading pair. An order is a tuple

$$o = (\mathsf{id},\ P,\ s,\ p,\ q_{\mathsf{rem}},\ t),$$

where id identifies the order, P the trader, $s \in \{\mathsf{buy}, \mathsf{sell}\}$ the side, p the limit price, q_{rem} the remaining unfilled quantity, and t the priority timestamp. The book state is $\mathsf{LOB} = (\mathsf{Bids}, \mathsf{Asks})$, with Bids sorted by decreasing price and Asks by increasing price, ties broken by t. Crossing orders are matched under this price–time priority; a match determines the executed quantity and the resulting obligations, and settlement then transfers the assets and updates or removes the affected orders. A large trading interest typically exceeds the liquidity resting at any single price level, and is therefore worked as several orders across a band of levels, executing against a sequence of counterparties over time—which is what makes it visible for long enough to be exploited.

b) The anatomy of parasitic trading.: Suppose Alice works a large buying interest by posting several sizeable buy orders across a band of price levels. Because q_{rem} is public on every one of them, a parasitic trader [2], [3] reads off them that Alice has strong latent buying demand that will persist across those levels. The adversary accumulates in the same direction, taking the liquidity Alice would otherwise have obtained; the book she faces is thinned, so to obtain execution she must bid higher. The adversary then supplies that liquidity back to her at the worse prices her own continued execution has produced. Alice bears an execution cost she would not have incurred had her size never been visible, and the adversary captures it.

Two features of this episode generalize, and both are about size rather than price. First, *the quantity is the signal*: side and limit price alone tell the adversary only that someone is willing to trade within that band, not the scale of the demand behind that willingness; it is q_{rem} that tells it whether the interest is worth attacking, how much inventory to commit, and how aggressively to move its own quotes. Second, *the attack has a size threshold*: positioning against Alice is not free—capital must be committed, quotes adjusted, liquidity provision forgone—and against orders of ordinary size these costs are not recovered, so the strategy is worthwhile only once the exposed quantity is large enough that the rent extracted exceeds the cost of extracting it.

Together these give the attack a necessary first step—*target selection*. Before positioning against anyone, the adversary must find a large resting order and distinguish it from the ordinary orders around it. This step is what the rest of the paper attacks: if no observer can tell a large order from a small one, no strategy conditioned on inferred size can outperform guessing, and the remainder of the pipeline has nothing to act on.

B. Conventional Countermeasures and Their Failure Modes

a) Withholding pre-trade information.: The most direct response is to stop publishing order information altogether. Conventional dark pools do exactly this, as do fully hiding DEX designs, which conceal balances, sides, prices, and quantities alike. The cost is borne by price formation. Comerton-Forde and Putniņš observe that a limit order displayed in a transparent book can influence prices before it executes, whereas the information in a dark order cannot enter public

prices until a trade occurs, and never does if the order goes unexecuted; they further find that a high level of non-block dark trading raises adverse-selection risk in the lit market and reduces informational efficiency, with the deterioration in their ASX sample beginning once dark trading in a stock exceeds roughly 10% of daily dollar volume [6]. For an order-book DEX whose market quality rests on public price formation, hiding the price dimension is therefore not a solution but a relocation of the damage.

b) Partial display and statistical inference.: Between full exposure and full darkness sits the iceberg order: only a peak is displayed while the remainder rests undisclosed, and as the peak executes a fresh tranche is replenished at the same price until the order is exhausted. Price and side stay public and continue to feed price formation, while the quantity behind them does not. The concealment, however, is incomplete, and the leak is mechanical: replenishment is itself observable, and the arithmetic relating displayed to executed volume constrains what lies behind it. Hiding is therefore not the same as being unpredictable. Bessembinder, Panayides and Venkataraman find that both the presence and the magnitude of hidden orders can be predicted to a significant, if imperfect, degree from observable order attributes, firm characteristics and market conditions [7], and detection algorithms operating directly on the book close the remaining gap: Zotikov and Antonov identify icebergs from inconsistencies between resting volume and the volume reported in trade summaries, together with the order modifications and fresh limit orders that follow a fill, then model the orders so identified to predict their total size, validated out of sample [10].

Concealment of this kind is therefore *obscurity rather than secrecy*: the quantity is never disclosed, yet remains estimable from the pattern the order's own execution leaves in the public record, and as further fills and replenishments are observed the order leaves behind ever more execution traces from which its hidden size can be inferred—precisely on the large, long-lived orders that are worth attacking. On chain the position is worse still. The adversary's raw material is the public record, and a blockchain supplies it in its most favourable form: the complete book history is permanently readable by anyone at no cost, at full granularity, replayable off-line and without the data-access asymmetries that limit who can run these inferences in a conventional venue. A defence whose strength is the empirical difficulty of an inference is weakest exactly where the inference is easiest to attempt.

C. The Design Point Invisibook Targets

The two subsections above bound the design space from opposite sides. Publishing the quantity hands target selection its input; hiding the price dimension protects the trader at the expense of the public price formation an order book exists to provide; and hiding the quantity by display policy alone yields a guarantee that degrades exactly on the orders that need it most. What the analysis leaves open is a design that concedes on neither axis: fully public along the price dimension, and protected along the size dimension by a guarantee that does not weaken as the order rests.

Invisibook occupies that point. Price, side, trading pair and fee stay in plaintext, so matching remains an ordinary deterministic priority rule that the chain executes and every node verifies in public, and every order continues to contribute to price formation exactly as in a lit book. Quantity, by contrast, is hidden *cryptographically* rather than by display policy: quantities and balances are carried as commitments, so concealment rests on the hiding property of the commitment scheme rather than on the empirical difficulty of inverting an execution pattern—and, since nothing is displayed and replenished, no such pattern is left behind to invert. The guarantee is therefore uniform in order size, and strongest where the threat is: a large order is indistinguishable from a small one no matter how long it rests or how much of the chain's history an adversary replays. This is the trade-off the remainder of the paper develops: privacy along the size dimension, preserved price discovery along the price dimension.

III. System and Threat Model

A. System Model

a) Entities and workflow.: Three entities appear in Invisibook. *Traders* place orders and, once matched, settle with each other; a matched pair is denoted A and B. *Blockchain nodes* maintain the ledger, execute the exchange's on-chain logic, and validate every state transition; since all exchange-side logic—order admission, matching, locking, and verification of settlement—is discharged by on-chain operations, there is no distinct operator entity in our setting. An *anonymous communication network* carries the point-to-point traffic between matched traders.

A trade proceeds in four steps. In *placement*, a trader submits an order publishing its trading pair, side, limit price and fee in plaintext while carrying its quantity only as a commitment, and collateralizes it by spending shielded funds. In *matching*, the chain pairs crossing orders under a deterministic public priority rule and locks the pair. In *off-chain co-proving*, the two matched traders run a secure two-party computation that compares their hidden quantities and jointly produces a proof of that computation, neither party learning the other's input. In *on-chain settlement*, the shares of the proof and of the result are submitted, assembled and verified by the chain, after which each party posts its own state update—order destruction, residual re-commitment, and the shielded transfer—each accompanied by a proof the chain verifies.

b) Building blocks.: Invisibook uses the following primitives as black boxes; none is a contribution of this work, and we defer their concrete instantiation to Section **??**.

- *Commitment.* A non-interactive commitment scheme $(\mathsf{Setup}, \mathsf{Com})$, where $\mathsf{Com}(m; r)$ commits to m under randomness r and an opening (m, r) is accepted if it reproduces the commitment. It is *hiding*—a commitment reveals nothing about m to a PPT adversary—and *binding*: no PPT adversary can open one commitment to

two distinct messages except with negligible probability. The blinding factor r is indispensable here, since order quantities live in a small, enumerable space and would otherwise fall to a dictionary attack.

- *Maliciously secure MPC.* A two-party protocol computing a function over private inputs while revealing only the prescribed output, secure against a party that deviates arbitrarily. We use SPDZ [12], built on additive secret sharing and information-theoretic MACs: any tampering with an input or an intermediate value is detected and the protocol aborts before any value is opened.
- *Collaborative zero-knowledge proof.* A zero-knowledge proof convinces a verifier that a public statement x admits a witness w without revealing w. Here the witness is split across the two traders and neither may reveal its share, so we use a collaborative zk-SNARK [11]: the proving algorithm is itself executed as an MPC protocol $\Pi(\mathsf{pp}, x, w_1, w_2) \rightarrow \pi$ over the parties' shares of w, while Setup and Verify are those of the underlying zk-SNARK. Beyond completeness, knowledge soundness and succinctness, it provides t-zero-knowledge: the view of an adversary controlling t of the provers is simulatable from the public input and the corrupted parties' own shares, so a corrupted prover learns nothing about the honest party's witness.
- *Shielded UTXO.* A Zcash-style private payment layer [13], [14]: note commitments accumulate in an append-only Merkle tree, a note is consumed by publishing a nullifier that is bound to it but unlinkable to its commitment, and spending is authorized by a zero-knowledge proof of tree membership, key knowledge and balance preservation. Amounts are confidential and spends unlinkable from the outputs they consume.
- *Digital signature.* A scheme $(\mathsf{Gen}, \mathsf{Sign}, \mathsf{Vrfy})$ [15] authenticating on-chain messages, so that a valid signature attests both the origin and the integrity of the message.

c) Infrastructure assumptions.: Invisibook is not self-contained: its guarantees are stated relative to properties it obtains from the layers beneath it, and a failure of either the ledger or the transport voids them regardless of how the protocol itself behaves. Of the underlying blockchain we assume:

(i) *Consensus safety and liveness.* Confirmed transactions are final, and valid transactions are eventually included.
(ii) *Integrity of execution.* On-chain operations execute exactly as written.
(iii) *Censorship resistance.* No adversary can indefinitely suppress a specific party's transactions.
(iv) *Confidential balances.* The chain supports an asset representation whose amounts are never published in plaintext and whose spends are unlinkable from the outputs they consume, as in the shielded UTXO layer above.

Assumptions (i) and (iii) are load-bearing rather than decorative: the timeout mechanisms of Section VI-D assign blame by the presence or absence of a message on chain by a deadline, so an adversary able to suppress an honest party's transaction could manufacture an apparent default and have that party penalized. Assumption (iv) is a precondition rather than a convenience: settlement moves value between the two parties, so on a chain with plaintext balances an observer reads the size of every fill directly off the balance delta—and with it the quantity of the smaller order, since the fill is that order in full. No concealment at the order layer survives a transparent balance layer beneath it, and a chain lacking (iv) cannot host Invisibook at all. Beyond balance confidentiality we assume the chain conceals nothing, and we assume no trusted hardware and no trusted operator.

Finally, settlement runs over a point-to-point channel between the two matched parties, and that channel is itself an attack surface: if the parties connect directly, network-level metadata such as IP addresses links a trader's network identity to its on-chain pseudonym, undoing at the network layer the privacy achieved at the protocol layer. Once a large trader is identified this way, an adversary can target it across subsequent sessions even without learning any individual order quantity. We therefore require that all peer-to-peer settlement traffic be routed over an anonymous communication network such as Tor [16]. The cryptographic guarantees of settlement do not depend on the transport, but the end-to-end privacy of the system is only as strong as the anonymity of the channel it runs over.

B. Threat Model

The adversary Invisibook faces has two components. They are not alternatives: a single adversary may occupy both roles simultaneously, and our guarantees are stated against that combined view. Corruption is static in both cases.

a) Passive public observers.: The first component observes public state only, and the capability that matters is inference from observation rather than protocol deviation. We deliberately admit the strongest possible observers. The *chain nodes acting as operator* hold the public chain state, which they may analyze arbitrarily; as noted above the operator role has no discretionary step at which to deviate, since all exchange-side logic is executed by on-chain operations that are public, deterministic and re-validated by every node. *External observers and arbitrageurs* may read the full chain history and may themselves submit orders to probe the book; their capabilities are exactly two, observe and trade honestly.

Restricting this class to passive adversaries is a consequence of the construction rather than an assumption of good behaviour. Every on-chain message—orders, proof shares, settlement updates—carries a signature under the submitter's key, so impersonation and tampering are excluded by the unforgeability of the signature scheme; and every on-chain step is executed autonomously and deterministically by the chain, leaving no point at which a party could deviate without producing an object the chain rejects. What remains available to this class, the chain nodes included, is precisely passive analysis of the public view.

b) Malicious co-computing counterparty.: The second component inverts this picture: it is the very counterparty taking part in the trade. There is no external server, no designated computation service, and no trusted third party, in contrast to prior privacy-preserving matching systems that rely on a centralized matcher or a designated computation service [5], [4]. The party against whom each trader must be protected is therefore its own co-computing counterparty—a participant in every step of the protocol, and thus the party best positioned to probe the other's private state during the interaction. Following the adversary model of collaborative proving [11], we consider an adversary that statically corrupts *one of the two parties* and additionally observes all on-chain data. With two participants and one possible corruption this is the *dishonest-majority* setting: privacy cannot rest on an honest majority, and must be guaranteed against a single corrupted party that may deviate arbitrarily.

This is why we require malicious rather than semi-honest security. Such a party may deviate in two ways, answered by different mechanisms. The first is *detectable deviation*: tampering with an input or an intermediate value violates the information-theoretic MAC checks of the secure computation, or produces an object that fails proof verification, and either way the protocol aborts before any value is opened. The second is *stalling*: a party may simply withhold messages, which no cryptographic check detects. Since a matched pair can no longer be cancelled while settlement is pending, the counterparty can only wait, its capital and its trading intent held inside a match that never completes. We therefore treat stalling as a security concern in its own right.

C. Security Goals

We state three goals: the privacy of resting orders, the legitimacy and correctness of the private computation, and security under malicious abort. All three must hold together. Order privacy without legitimacy and correctness lets a counterparty misreport a fill and steal value under cover of the very commitments that protect it; correctness without order privacy protects a trade whose size was already public; and without abort security, an adversary that can neither read nor falsify can still drag settlement out indefinitely, costing its counterparty the trading opportunities it would otherwise have taken.

a) What is hidden and what is public.: Three classes of value must never appear in plaintext on chain. The *order quantity* is the signal target selection consumes directly, and Invisibook conceals it behind a commitment carried in the order. The *account balance* is a target in the same sense: an adversary that identifies a large balance knows the trader has latent capacity to deploy and can position for it in advance of any order; this class is discharged by assumption (iv) rather than by any mechanism of ours. The *residual quantity of a partially filled order* must be hidden because a large order that survives its first fill re-enters the book; publishing its remainder would restore it as a visible target at exactly the moment it becomes one again. These are not independent secrets—the residual is the difference of the two matched quantities, and every fill moves the balances accordingly—but distinct surfaces through which the same quantities would emerge, so publishing any one defeats the concealment of the others.

Two categories are deliberately public. The first is everything public matching consumes—trading pair, side, limit price and fee; concealing these is precisely what we set out *not* to do [6]. The second is the comparison result b, which discloses which of two matched orders is the smaller, or that they are equal. Publishing b is not a concession extracted by the construction but a recognition of what an order book unavoidably reveals: a fully filled order is removed from the book, and that removal is a public event, so the relative order of the two matched quantities is disclosed by order-book semantics whatever the protocol does. Crucially b carries no magnitude—it orders two quantities without bounding either—and, by the spend unlinkability the chain provides, the results of distinct settlements cannot be linked into a trajectory of any one party's position. Its disclosure therefore does not restore target selection.

1) Goal 1: Privacy of resting orders: This goal answers the passive adversaries above, whose view is the entire ledger over its entire history: however they observe that history and whatever other data they draw on, order quantities must remain impossible to infer.

Definition 1 (Hidden state and public transcript)**.** The *hidden state* of an execution is the tuple $S = (\vec{q}, \vec{v}, \vec{q}^{\,\mathrm{res}})$, comprising the order quantities, the shielded balance amounts, and the residual quantities of partially filled orders. The *public transcript* T comprises the order metadata (pair, side, limit price and fee), the matching sequence, the fill and destruction markings on the book, and the comparison results $\{b_i\}_i$.

Definition 2 (Amount indistinguishability)**.** Consider two executions with identical public transcripts T but different hidden states S and S'. The system satisfies *amount indistinguishability* if no polynomial-time observer can tell which of the two it is looking at, where what the observer sees is everything on chain: the amount-hidden order book, the shielded balance state, and all comparison and settlement messages.

This neutralizes the attack pipeline at its first step. No polynomial-time observer can distinguish a book containing a large order from one containing a small order, so no strategy conditioned on inferred size achieves an advantage over guessing: an adversary watching the chain cannot tell Alice's order from any other order at price p, and therefore cannot decide whether positioning against it would repay the cost of doing so.

2) Goal 2: Legitimacy and correctness of the private computation: Matching settles only the price; the quantity-dependent work is left to a private computation between the two traders, every input to which exists only as a commitment. What must be guaranteed is therefore not merely that the computation is right, but that it was performed on the right values.

Property 1 (Legitimacy and correctness)**.** An accepted settlement satisfies, except with negligible probability: (i) the compared values are exactly the quantities opening the two matched orders' on-chain commitments; (ii) the private computation that produced the comparison result b is itself correct and untampered—b was genuinely computed jointly by the two parties, and neither can unilaterally tamper with that computation or with its output; (iii) the residual commitment commits to the difference of the two quantities, which is non-negative; and (iv) the newly created shielded outputs reflect a transfer of exactly the filled amount.

Clause (ii) is stated separately because it is what makes the guarantee robust in the dishonest-majority setting: without it a corrupted party could recover the true result locally while corrupting the published one—the truth known to it alone, the public record unilaterally spoiled.

Property 2 (Scope of disclosure)**.** Beyond b, a settlement discloses: to the larger party, the filled amount; to the smaller party, nothing beyond the fact that the counterparty's quantity exceeds its own; and to the chain and all third parties, nothing.

The reach of the filled amount deserves a word. Since it ordinarily equals the smaller order in full, the larger party's learning it is the same as learning the smaller party's order quantity; but that order has at this moment been fully filled and has left the book, so its holder no longer has any resting order to be anticipated, and the knowledge cannot be converted into an advantage against its subsequent trading. In the other direction, the smaller party learns only that the counterparty's quantity exceeds its own, while the residual the larger party actually needs protected remains behind a fresh commitment. Privacy during the computation itself is inherited from the secure computation: the secret sharing is information-theoretically hiding and the MACs ensure any deviation is detected before any value is opened, so a corrupted party's view is simulatable from its own input together with b.

3) Goal 3: Security under malicious abort: The two goals above are of the security-with-abort flavour, and abort is not a neutral outcome here. They prevent an adversary from reading its counterparty's size and from submitting false data, but leave it free to drag settlement out and never complete. A matched pair can no longer be cancelled while settlement is pending, so the compliant party can only keep waiting, its capital and its trading intent held inside a match that never closes while other opportunities pass by. This is a cheap attack and an undetectable one: withholding a message is not a deviation any cryptographic check can catch. Abort security is therefore a security requirement in its own right, not an availability footnote.

Property 3 (Security under malicious abort)**.** When a malicious party unilaterally withholds or delays a required message, the protocol guarantees that the behaviour is unprofitable to it, and that the trading-opportunity cost it inflicts on the honest counterparty is eliminated as far as possible.

IV. Preliminaries

This section introduces the blockchain and cryptographic background used in our construction.

A. On-Chain States and Operations

A blockchain can be viewed as a public ledger that maintains on-chain states and validates state transitions. Decentralized exchanges are application-layer protocols built on top of this ledger infrastructure, using on-chain operations to represent, update, and settle trading states.

1) UTXO Model.: The UTXO model consists of UTXOs and transactions. A *UTXO* is an unspent transaction output that represents a spendable asset together with its spending condition, formally denoted by

$$u = (r, a, v, \phi),$$

where $r = (\mathsf{txid}, j)$ is the output reference, indicating that this UTXO is the j-th output created by transaction txid; $a \in \mathcal{A}$ is the asset type; $v \in \mathbb{V}$ is the asset value; and $\phi \in \Phi$ is the spending predicate specifying the condition under which the output can be consumed.

A transaction consumes existing UTXOs as inputs and creates new UTXOs as outputs, defined as follows:

$$T = (\mathsf{in}, \mathsf{wit}, \mathsf{out}),$$

where $\mathsf{in} = (r_1, \ldots, r_m)$ is a sequence of input references to existing UTXOs, $\mathsf{wit} = (w_1, \ldots, w_m)$ is a sequence of witnesses, and $\mathsf{out} = ((r'_1, a'_1, v'_1, \phi'_1), \ldots, (r'_n, a'_n, v'_n, \phi'_n))$ is a sequence of newly created outputs. Each output in out obtains a fresh reference $r'_j = (H(T), j)$, where H is a collision-resistant hash function.

Given a ledger state L, a transaction T is valid if the following conditions hold:

1) Every input reference r_i refers to an unspent output.

$$u_i = (r_i, a_i, v_i, \phi_i) \in \mathsf{UTXO}(L).$$

2) Each witness satisfies the corresponding spending predicate.

$$\phi_i(T, w_i) = 1 \qquad \text{for all } i \in [m].$$

3) Value is conserved for each asset type. That is, for every $a \in \mathcal{A}$,

$$\sum_{i:a_i=a} v_i \geq \sum_{j:a'_j=a} v'_j.$$

The difference between the two sides represents transaction fees or burned value.

A valid transaction updates the ledger state by removing the consumed outputs and adding the newly created outputs:

$$\mathsf{UTXO}(L') = \mathsf{UTXO}(L) \setminus \{(r_i, a_i, v_i, \phi_i)\}_{i=1}^{m} \cup \{(r'_j, a'_j, v'_j, \phi'_j)\}_{j=1}^{n}.$$

2) Decentralized Exchanges: A decentralized exchange (DEX) is a distributed-ledger-based protocol or application that enables users to trade crypto assets without relying on a centralized exchange operator as either the trading intermediary or asset custodian [1]. At a high level, a DEX defines application-layer trading states and operations on top of the underlying ledger. These states are ultimately represented by ledger states, and the operations are implemented as ledger transactions that update those states.

B. Cryptography

A zero-knowledge proof allows a prover to convince a verifier that a public statement x satisfies a relation $\mathcal{R}$ with respect to some private witness w, without revealing information about w beyond the validity of the statement. In our system, since order amounts are hidden on-chain, the matched buyer and seller jointly submit the updated state along with a proof attesting to the correctness of the state-transition computation. Once the on-chain verifier accepts the proof, the updated state is finalized.

However, the witness required to generate the proof is not held by a single party. For instance, the party placing the larger order should not reveal its order amount to the party placing the smaller order. Therefore, we utilize the collaborative zero-knowledge proof protocol to generate the proof while preserving the privacy of each party's private input.

1) Collaborative Zero-Knowledge Proof: [17], [18], [19]

Definition 3. Let N represent the number of servers, and $S_1, \ldots, S_N$ be the servers. Let $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ be a zk-SNARK for some NP relation $\mathcal{R}$. Let x be the public input and w be the witness. For each server S_i where $i \in [N]$, w_i is the packed shares of w received by S_i. A collaborative zk-SNARK for an NP relation $\mathcal{R}$ consists of a tuple of algorithms $(\mathsf{Setup}, \Pi, \mathsf{Verify})$, where:

- $\mathsf{Setup}(1^\lambda, \mathcal{R}) \to \mathsf{pp}$: This is the same as the setup algorithm Setup of the underlying zk-SNARK. It takes the security parameter λ and the NP relation $\mathcal{R}$ as inputs and outputs the public parameter pp.
- $\Pi(\mathsf{pp}, x, w_1, \ldots, w_N) \to \pi$: This is an MPC protocol among N servers, and it computes the prover algorithm Prove of the underlying zk-SNARK. Given the public parameters pp, the public statement x and the servers' received packed secret shares $w_1, \ldots, w_N$, the servers engage in Π and collaboratively generate a proof π.
- $\mathsf{Verify}(\mathsf{pp}, x, \pi) \to \{0, 1\}$: This is the same as the verification algorithm Verify of the underlying zk-SNARK. It takes the public parameter pp, the statement x, and the proof π as inputs and outputs a bit b indicating acceptance $(b = 1)$ or rejection $(b = 0)$.

This framework is secure if it satisfies the following properties:

- **Completeness.** For all $(x, w) \in \mathcal{R}$, the following relation holds:

$$\Pr\begin{bmatrix}\mathsf{Verify}(\mathsf{pp}, x, \pi) = 1 : \\ \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}), \\ \pi \leftarrow \Pi(\mathsf{pp}, x, w_1, \ldots, w_N)\end{bmatrix} = 1. \tag{1}$$

- **Knowledge Soundness.** For all x, and all sets of PPT algorithms $\widetilde{\mathsf{S}} = \{\widetilde{S}_1, \ldots, \widetilde{S}_N\}$, there exists a PPT extractor $\mathcal{E}$ such that,

$$\Pr\begin{bmatrix}\mathsf{Verify}(\mathsf{pp}, x, \pi^*) = 1 : \\ \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}), \\ w^* \leftarrow \mathcal{E}^{\widetilde{\mathsf{S}}}(\mathsf{pp}, x), \\ \pi^* \leftarrow \widetilde{\mathsf{S}}(\mathsf{pp}, x), \\ (x, w^*) \notin \mathcal{R}\end{bmatrix} \leq \mathsf{negl}(\lambda). \tag{2}$$

- **t-zero-knowledge.** For all PPT adversary $\mathcal{A}$ controlling at most t servers denoted as Corr, $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R})$, there exists a simulator $\mathcal{S}$ such that for all x, w, where $b \leftarrow \mathcal{R}(x, w) \in \{0, 1\}$, the following relation holds:

$$\mathsf{View}^{\Pi}_{\mathcal{A}}(x, w) \approx \mathcal{S}\left(\mathsf{pp}, x, b, \{w_i\}_{i \text{ s.t. } S_i \in \mathsf{Corr}}\right). \tag{3}$$

  Here $\mathsf{View}^{\Pi}_{\mathcal{A}}(x, w)$ denotes the view of $\mathcal{A}$ from the real-world execution of Π and $\mathcal{S}(\mathsf{pp}, x, b, \{w_i\}_{i \text{ s.t. } S_i \in \mathsf{Corr}})$ is the view generated by $\mathcal{S}$ given x and inputs from corrupted parties. We use $\approx$ to denote the two distributions are computationally indistinguishable.
- **Succinctness.** The proof size and verification time are both of $\mathsf{poly}(\lambda, |x|, \log |w|)$.

In our applications, the number of parties $N = 2$. Therefore, we can either use secure multiparty computation (MPC) and homomorphic encryption (HE) as two instantiations of the protocol $\Pi(\mathsf{pp}, x, w_1, w_2)$.

a) Collaborative ZK Instantiated by MPC: [Tianyi: add this after more survey.]

2) Commitment:

a) Commitment.: A non-interactive commitment scheme is a pair of randomized algorithms $\Pi = (\mathsf{SetupCommit}.$ The setup algorithm $\mathsf{Setup}1^\lambda)$ outputs public parameters pp, and the commitment algorithm $\mathsf{Commit}pp, m; r)$ outputs a commitment c to message m using randomness r. An opening (m, r) is accepted if $c = \mathsf{Commit}pp, m; r)$.

A commitment scheme satisfies two standard security properties.

- *Hiding* requires that the commitment c reveals no information about m: for any two messages m_0, m_1, no PPT adversary can distinguish a commitment to m_0 from a commitment to m_1 except with negligible advantage.
- *Binding* requires that no PPT adversary can find a commitment c that admits two valid openings (m_0, r_0) and (m_1, r_1) with $m_0 \neq m_1$, except with negligible probability.

In Sections 4 and 5, we treat the above cryptographic mechanisms as idealized black-box functionalities and use

them to describe the protocol logic independently of any concrete implementation. In Section 6, we instantiate these functionalities with concrete cryptographic constructions and discuss their efficiency and deployment trade-offs.

3) Digital Signature: A digital signature scheme [15] is a cryptographic primitive for authenticating messages. It is defined by three algorithms: Gen generates a public/secret key pair, Sign produces a signature on a message using the secret key, and Vrfy checks the signature using the public key. Its goal is to ensure message integrity and authenticity: a valid signature convinces verifiers that the message was signed by the secret-key holder and was not modified.

V. On-Chain State and Operations

This section specifies the on-chain data structures of Invisibook and the life cycle of a trade as observed by the blockchain: order submission, matching, and the verification of settlement results. The off-chain settlement protocol itself—a secure multi-party computation between the matched counterparties—is only summarized here at the interface level; its full construction is deferred to Section VI. Throughout, we write $\mathsf{Com}(x; r)$ for a commitment to the message x (possibly a tuple) under randomness r, and $[P]$ for the truth value of predicate P.

A. Amount-Hiding UTXO

Invisibook adopts a UTXO-based account model in which, exactly as in Zcash, *nothing* about a UTXO appears in plaintext on-chain. Formally, the content of a UTXO is a tuple

$$\mathsf{utxo} = \big(\mathsf{pk},\ \mathsf{assetID},\ v\big), \qquad \mathsf{cm} = \mathsf{Com}(\mathsf{utxo};\ r),$$

where pk is the owner's public key, assetID identifies the asset type, and v is the amount. The chain records only the hiding and binding commitment cm: neither the owner's key, nor the asset type, nor the amount is ever published. Ownership and asset-type validity are instead verified in zero-knowledge at spending time, and binding assetID inside the commitment ties each amount to its asset type, so that a proof over committed amounts of one asset can never be reused for or mixed with another. Only the owner knows the opening (utxo, r). The randomness r (a blinding factor) is indispensable: amounts live in a small, enumerable space, so without r any observer who knows or guesses pk and assetID could precompute the commitments of all candidate amounts and recover v by lookup—a rainbow-table-style dictionary attack.

Our UTXO definition deliberately mirrors that of Zcash [13], [14], and we refer to these as *private UTXOs*: commitments are accumulated in an append-only Merkle tree, and a UTXO is consumed by publishing a *nullifier* that is cryptographically bound to the note but unlinkable to its commitment. Spending is authorized by a zero-knowledge proof of (i) membership of the note commitment in the Merkle tree, (ii) knowledge of the spending key, and (iii) balance preservation over committed amounts. The privacy guarantees of this construction—amount confidentiality via commitment hiding and spend/output unlinkability via nullifiers—have been formally established in the Zerocash security analysis [13]; Invisibook inherits these guarantees directly and we therefore do not re-prove them. In particular, they ensure that no on-chain observer can trace or analyze any individual UTXO—large UTXOs included—which is precisely the property we need: it denies the attacker the ability to locate large holdings at the UTXO level.

We stress that using Zcash to develop the account-layer privacy in this section is only for concreteness and ease of exposition. The requirement Invisibook places on the underlying blockchain (stated earlier) is that account balances must be hidden; Zcash is merely one construction that meets it. Any protocol achieving the same effect—hiding amounts while keeping UTXOs untraceable and unlinkable—is equally applicable, and Invisibook is not tied to Zcash.

B. Amount-Hiding Order

An order is a *special UTXO*: unlike an ordinary private UTXO, whose entire content is hidden inside its commitment, an order deliberately publishes every attribute that public matching requires, hiding only its quantity. Formally, an order is a tuple

$$o = \big(\mathsf{oid},\ \tau,\ d,\ p,\ \mathsf{cm}_q,\ \mathsf{pk},\ f\big), \qquad \mathsf{cm}_q = \mathsf{Com}(q;\ r_q),$$

where oid is a unique order identifier, τ the trading pair, $d \in \{\mathsf{buy}, \mathsf{sell}\}$ the side, p the limit price *in plaintext* (or a market-order flag), cm_q a commitment to the order quantity q (with r_q blinding the commitment exactly as r does for notes), pk the owner's key—published so that the chain can authenticate the owner's subsequent messages, namely its proof share and its settlement update—and f the *plaintext* fee carried by the order. The fee is denominated in the chain's native token and its amount is public by design, since on-chain matching must consult fee sizes (see the priority rules below). The block height at which the order transaction is included and the transaction's index within its block are properties of the enclosing transaction rather than of the order itself; as public chain metadata they nevertheless participate in matching priority, as described below.

Creating an order consumes private UTXOs of total value $p \cdot q + f$. Exactly as in Zcash, the submitter destroys the input UTXOs by publishing their nullifiers—the nullifier mechanism is what guarantees that the spent UTXOs cannot be traced or analyzed—and generates the corresponding zero-knowledge proof and metadata, attesting that the committed total of the destroyed UTXOs equals the committed value $p \cdot q$ plus the plaintext fee f; that is, the order is fully collateralized. The fee is paid to the miner in the same way transaction fees are handled in Zcash: the fee amount is public while every other value remains hidden. Only after the chain verifies this spend is o admitted into the order book; thus the book never contains uncollateralized orders, even though no verifier learns any quantity.

C. Trade Flow

a) Matching.: Since prices are public, matching requires no cryptographic machinery: the chain runs a conventional

deterministic price–time priority algorithm directly over the plaintext price field. Concretely, crossing orders are matched under the following priority rules:

1) *Price priority.* Among crossing orders, the best-priced order is matched first.
2) *Time priority.* Ties in price are broken by the block height at which the order transaction was included: orders in earlier blocks are matched first.
3) *Fee priority.* If tied orders share both price and block height, they are matched in descending order of the plaintext fee f carried by the order.
4) *Intra-block index.* If price, block height, and fee all coincide, the order whose transaction has the smaller `transaction_index` within the block is matched first.

A structural consequence of amount hiding is that matching is strictly *pairwise*: each match consists of exactly one bid and one ask. In a transparent order book, the matching engine fills a large incoming order by walking the book and aggregating multiple resting orders until the incoming quantity is exhausted. This is impossible here—the chain cannot directly compare, split, or sum quantities it only sees as commitments, and so it cannot even determine *when* an incoming order is exhausted. A match in Invisibook is therefore a *price match* rather than a full-fill match: the chain pairs the incoming order with the single highest-priority crossing order, and all quantity-dependent computation is deferred to settlement. Once a pair is matched, the two orders are immediately locked on-chain: they can no longer be canceled and are forced into the settlement process.

b) Off-chain comparison and on-chain verification.: Settlement begins by privately confirming which of the two orders has the smaller quantity. The matched parties A and B execute a secure multi-party computation off-chain (Section VI) which, entirely inside the protocol, (i) verifies that each party's private input opens its on-chain quantity commitment, and (ii) compares the two quantities. The computation reveals nothing beyond a single bit $b = [\, q_A \leq q_B \,]$ identifying the smaller order (if the quantities are equal, both orders are fully filled). Alongside b, a zero-knowledge proof is collaboratively generated: each party submits its own share of the co-ZK proof to the chain, where the shares are assembled into a single complete co-ZK proof π_{cmp} (the construction of the shares and of their assembly is given in Section VI), attesting that b is the correct comparison of the values committed in cm_{q_A} and cm_{q_B}. The chain then verifies π_{cmp} against the locked orders' commitments. Only after this on-chain verification succeeds does the protocol proceed.

c) State updates.: Suppose without loss of generality that A holds the smaller order. Once the comparison has been verified on-chain, the parties complete the remaining interaction of the off-chain settlement protocol—through which the larger side learns the fill quantity; how is specified in Section VI—and each then submits its own state update to the chain:

- *Smaller side (A).* A's order, being fully filled, is *marked as destroyed* directly on the order book. Orders are public book entries, so this destruction is an explicit public marking and does *not* go through the nullifier mechanism. Alongside the destruction, a new private UTXO is created and paid to the counterparty B—a fresh note commitment whose committed amount corresponds to executing the full committed quantity q_A at the match price p—together with a proof π_A of this correspondence.
- *Larger side (B).* B's order is likewise *marked as destroyed*, and two objects are created in its place: a *new order* o'_B carrying the same pair, side, and price, whose quantity commitment $\mathsf{cm}'_{q_B} = \mathsf{Com}(q_B - q_A;\, r'_B)$ commits to the residual quantity under fresh randomness r'_B, and a new private UTXO paid to the counterparty A. These are accompanied by a proof π_B attesting that (i) the residual equals $q_B - q_A$ and is non-negative, (ii) the q_A used in this computation opens A's on-chain commitment cm_{q_A}—so B cannot understate the fill by substituting a fabricated counterparty quantity—and (iii) the committed amount of the new UTXO is consistent with the fill.

The chain verifies each proof against the current on-chain commitments and, upon success, applies the updates: both original orders are destroyed on the book, the residual order o'_B enters the book, and the newly created UTXOs join the private UTXO set as ordinary note commitments, indistinguishable from any other note. Any later spend of these UTXOs goes through the standard Zcash-style nullifier mechanism, so they cannot be traced or analyzed in subsequent operations.

At no point does the chain—or any observer—learn a plaintext quantity: matching operates on public prices, and settlement verification operates on commitments and a single comparison bit whose disclosure is inherent to order-book semantics (the destruction marking of a fully filled order is publicly observable regardless).

VI. Offchain Privacy Settlement

Matching only produces a *price match*: because order sizes are hidden behind commitments, the on-chain matching engine cannot tell whether the two matched quantities are equal, nor which is larger. The actual size comparison, the residual computation, and the state update are therefore deferred to an off-chain privacy-preserving settlement between the two matched parties, whom we denote A and B (Alice and Bob). Settlement must transition both parties' private states and be publicly verifiable on chain, while revealing to neither party—nor to any observer—more about the counterparty's order size than the settlement outcome inherently requires.

The core difficulty is the size comparison. Each order size is recorded on chain as a hiding and binding commitment $\mathsf{cm}_q = \mathsf{Com}(q, r)$, where q is the plaintext quantity and r a blinding factor, both known only to the order's owner. Neither party may reveal its plaintext quantity to the other in order to compare, since that would leak precisely the private value settlement is meant to protect. Indeed, probing the counterparty's quantity during settlement is the central adversarial behavior the threat model of Section III-B must capture. We resolve this with a maliciously secure two-party computation

that compares the two committed quantities without either party learning the other's input.

We stress that the two computing parties are *the traders themselves*: the matched pair A and B jointly run the secure comparison, each supplying its own private order data as input. There is no external server, no designated computation service, and no trusted third party. Consequently the party against whom each trader must be protected is its own co-computing counterparty. This mirrors the setting of collaborative zero-knowledge proofs [11], in which a proof over a distributed witness is produced jointly by the witness holders without any of them learning the others' shares; here the distributed witness is the pair of committed order sizes, and the secure computation is instantiated with SPDZ-style secret-sharing MPC [12].

A. Secure Comparison

The first phase of settlement determines which order is smaller. Let q_A, r_A (resp. q_B, r_B) be A's (resp. B's) plaintext quantity and blinding factor, and let $\mathsf{cm}_{q_A}, \mathsf{cm}_{q_B}$ be the corresponding on-chain commitments. A and B run a two-party MPC that, entirely under secret sharing, evaluates the comparison

$$b = \mathsf{sign}(q_A - q_B) \in \{-1, 0, 1\}, \tag{4}$$

where $b = -1$ means $q_A < q_B$ (A is the smaller order), $b = 0$ means $q_A = q_B$ (the two orders are equal), and $b = 1$ means $q_A > q_B$ (B is the smaller order).

The comparison alone says nothing about what the parties actually fed into the computation. A malicious party could supply a fabricated quantity and thereby manipulate b. We therefore use a collaborative zero-knowledge proof (co-ZK) [11] to guarantee the *legitimacy of the MPC inputs*: to prove that the inputs to the computation are exactly the order quantities the parties previously committed to on chain. Concretely, the two parties *jointly generate*, inside the MPC, a publicly verifiable zero-knowledge proof π_{cmp}. Let share_A^b and share_B^b be the two additive shares of the comparison result b output by the MPC, and let $\mathsf{cm}_A^b = \mathsf{Com}(\mathsf{share}_A^b, \rho_A)$ and $\mathsf{cm}_B^b = \mathsf{Com}(\mathsf{share}_B^b, \rho_B)$ be commitments to those shares, where ρ_A and ρ_B are blinding factors. The statement of π_{cmp} is that there exist (q_A, r_A), (q_B, r_B), b, $(\mathsf{share}_A^b, \rho_A)$ and $(\mathsf{share}_B^b, \rho_B)$ such that

$$\pi_{\mathrm{cmp}} : \begin{cases} \mathsf{Com}(q_A, r_A) = \mathsf{cm}_{q_A}, \\ \mathsf{Com}(q_B, r_B) = \mathsf{cm}_{q_B}, \\ b = \mathsf{sign}(q_A - q_B), \\ \mathsf{share}_A^b + \mathsf{share}_B^b \equiv b \pmod p, \\ \mathsf{cm}_A^b = \mathsf{Com}(\mathsf{share}_A^b, \rho_A), \\ \mathsf{cm}_B^b = \mathsf{Com}(\mathsf{share}_B^b, \rho_B). \end{cases} \tag{5}$$

Its public inputs are $\mathsf{cm}_{q_A}, \mathsf{cm}_{q_B}, \mathsf{cm}_A^b, \mathsf{cm}_B^b$; the comparison result b itself and both shares remain witnesses and are not disclosed in the first step.

The first three constraints capture *input legitimacy and correctness of the comparison*—clause (i) of Property 1: the values compared are exactly the quantities that open the on-chain commitments $\mathsf{cm}_{q_A}, \mathsf{cm}_{q_B}$, and b is the correct result of comparing them. The last three *pin down the shares*: the two shares really do sum to that legitimate b, and they open the public commitments cm_A^b and cm_B^b respectively. Because π_{cmp} is produced jointly by the two parties inside the MPC, with neither learning the other's witness share, it simultaneously anchors the MPC inputs to the on-chain commitments and fixes in advance the shares either party may submit in the second step—without revealing q_A, q_B, or even b—rendering both guarantees publicly verifiable by the chain and every third party.

The MPC output is not revealed to either party in the clear; it is produced as additive shares—of both the comparison result b and the jointly generated proof π_{cmp}. Party A holds share_A^b and share_A^π, party B holds share_B^b and share_B^π, where

$$\mathsf{share}_A^b + \mathsf{share}_B^b \equiv b \pmod p,$$

and share_A^π together with share_B^π combine to the complete π_{cmp}. Crucially, both kinds of share *must be submitted on chain*: the parties reconstruct neither of them between themselves, and both b and π_{cmp} are assembled only on chain, in two successive steps (Section VI-B). Until then, what each party holds is an uninformative share.

We use SPDZ [12], a maliciously secure MPC framework built on additive secret sharing and information-theoretic MACs (IT-MACs), as the secure computation underlying the co-ZK. The IT-MACs guarantee that if either party tampers with its input or with any intermediate value, the deviation is detected by the other party and the protocol aborts before any value is opened.

B. On-Chain Share Combination

Submission and assembly proceed in two steps whose order cannot be reversed: the proof is assembled and verified first, and only then is the comparison result assembled. Together these two steps discharge clause (ii) of Property 1—that the computation producing b is itself correct and untampered, and that neither party can unilaterally spoil its output.

Step 1: submit proof shares and share commitments; assemble and verify π_{cmp}. Within Δ_{sub} block times ($\Delta_{\mathsf{sub}} = 10$ in our instantiation), each party submits two items: its proof share, and the commitment to its own share of the comparison result—that is, A submits $\left(\mathsf{share}_A^\pi,\ \mathsf{cm}_A^b\right)$ and B submits $\left(\mathsf{share}_B^\pi,\ \mathsf{cm}_B^b\right)$. These commitments are public inputs of π_{cmp} and are *not* contained in the proof shares, so they must be supplied explicitly alongside the shares for the chain to verify the proof at all.

The chain first assembles the two proof shares into the complete proof

$$\pi_{\mathrm{cmp}} \leftarrow \left(\mathsf{share}_A^\pi,\ \mathsf{share}_B^\pi\right),$$

and then verifies π_{cmp} against the public inputs $\mathsf{cm}_{q_A}, \mathsf{cm}_{q_B}, \mathsf{cm}_A^b, \mathsf{cm}_B^b$. A successful verification establishes

two things at once: that the compared inputs are the legitimate quantities opening $\mathsf{cm}_{q_A}, \mathsf{cm}_{q_B}$, and that the two share commitments are now fixed on chain as the reference against which the shares are checked in the second step. No trust is needed in the submitted commitments: a proof is generated for one specific set of public inputs, so any substituted cm_i^b makes verification fail immediately. If verification fails, the settlement terminates here: the shares of the comparison result are never submitted, and b is never reconstructed.

Step 2: submit the shares and assemble b. Only after the on-chain verification of Step 1 succeeds do the parties submit their shares of the comparison result, within a further Δ_{sub} block times. Crucially, each party must *accompany* its share with a zero-knowledge proof that the submitted share is indeed the result of the very computation attested by the already-verified π_{cmp}: A submits $\big(\mathsf{share}_A^b,\ \pi_{\mathrm{bind}}^A\big)$ and B submits $\big(\mathsf{share}_B^b,\ \pi_{\mathrm{bind}}^B\big)$, where

$$\pi_{\mathrm{bind}}^i: \quad \exists\, \rho_i \ \text{ s.t. } \ \mathsf{cm}_i^b = \mathsf{Com}\big(\mathsf{share}_i^b,\, \rho_i\big), \qquad i \in \{A, B\}. \tag{6}$$

That is, the submitted share does open the commitment cm_i^b that was fixed on chain alongside π_{cmp} in Step 1. The chain verifies π_{bind}^A and π_{bind}^B, and only once both pass does it add the shares,

$$b = \mathsf{share}_A^b + \mathsf{share}_B^b \bmod p,$$

and publish b, which triggers the remainder of settlement.

This ordering, together with the binding proof, is deliberate, and its necessity follows from a concrete attack. Suppose A is malicious and B honest. If both kinds of share were submitted together and the chain simply added them to recover b, A could submit a forged share_A^b. Being honest, B submits its true share_B^b, which becomes visible to A the moment it appears on chain; A then adds it to the true share it actually holds and recovers the *real* b locally. The chain, meanwhile, adds B's true share to A's forged one and obtains a wrong b. The outcome is an asymmetry entirely in the adversary's favour: A alone learns the true comparison result, while the public on-chain result has been unilaterally corrupted—precisely the failure clause (ii) of Property 1 rules out.

The two-step ordering together with π_{bind} closes this path. When π_{cmp} is verified in Step 1, the share commitments $\mathsf{cm}_A^b, \mathsf{cm}_B^b$ are already fixed on chain as its public inputs, and the proof guarantees that the shares they commit to do sum to the legitimate b. In Step 2, a party wishing to submit a forged share would have to produce a verifying π_{bind} for that forged value—equivalent to forging a commitment opening for a value it never committed to—which, by the binding property of the commitment scheme and the soundness of the proof system, is computationally infeasible. A forged share is therefore rejected outright on chain rather than silently combining into an incorrect b.

Assembling on chain rather than reconstructing between the parties makes the finality of b and π_{cmp} independent of either party: neither can unilaterally determine or forge the outcome. If a party fails to submit the corresponding share within the deadline of either step, the liveness mechanism of Section VI-D applies.

C. Settlement

Once b is public, the two parties settle asymmetrically according to who holds the smaller order. Without loss of generality assume $b = -1$, so A is the smaller order and B the larger; the case $b = 1$ is symmetric with the roles exchanged, and the case $b = 0$ is treated below.

Smaller party (A): reveal and finalize. A sends its plaintext quantity q_A and blinding factor r_A to B over the secure point-to-point channel. This is the disclosure the outcome inherently requires: the traded amount equals the smaller order's full quantity, so once A is known to be smaller, B must learn q_A to settle. A then updates its on-chain state, marking its order *fully filled* and submitting the shielded-balance update reflecting the settled q_A, following the on-chain state update procedure of Section V-C.

Larger party (B): verify, compute residual, re-commit. On receiving (q_A, r_A), B first checks locally that

$$\mathsf{Com}(q_A, r_A) = \mathsf{cm}_{q_A},$$

confirming that the plaintext A sent matches A's on-chain commitment and that A is not cheating. B then computes its residual quantity

$$q'_B = q_B - q_A,$$

samples a fresh blinding factor r'_B, and forms a new order commitment $\mathsf{cm}'_{q_B} = \mathsf{Com}(q'_B, r'_B)$, so that the residual never appears in plaintext at any point. Finally B submits to chain: the updated order commitment $\mathsf{cm}_{q_B} \to \mathsf{cm}'_{q_B}$ (the residual order remains resting in the book), the corresponding shielded-balance update following the same procedure, and a zero-knowledge proof π_B attesting

$$\pi_B: \begin{cases} q'_B = q_B - q_A, & \text{(residual computed correctly)} \\ \mathsf{Com}(q_A, r_A) = \mathsf{cm}_{q_A}. & (q_A \text{ opens } A\text{'s commitment}) \end{cases} \tag{7}$$

Statement (1) proves B did not tamper with the residual arithmetic; statement (2) proves that the q_A used by B is exactly the value A committed to on chain, preventing B from fabricating a smaller counterparty quantity to its own advantage. Together with the balance-update proofs of Section V-C, these discharge clauses (iii) and (iv) of Property 1; the non-negativity of the residual required by clause (iii) follows from the publicly verified b, which already certifies $q_B \geq q_A$.

Equal quantities ($b = 0$). When $q_A = q_B$, both orders are fully filled: the residual $q'_B = 0$, B's order is likewise marked fully filled, and each party can already infer that the counterparty's quantity equals its own.

The disclosure here is deliberate and minimal, and matches Property 2. The larger party learns the smaller party's quantity, which is exactly the traded amount—the minimum any settlement functionality must reveal—while the smaller party learns only that the counterparty's quantity exceeds its own, exactly

the leakage already implied by the public value b. Neither the chain nor any third party learns either quantity.

D. Liveness and the Challenge Mechanism

This subsection specifies the mechanism realizing Property 3. Our protocol provides security with abort: a malicious counterparty cannot violate correctness or privacy, but it can stall settlement—not by deviating detectably, but by simply withholding messages. Since a matched pair is locked while settlement is pending, withholding would otherwise let a malicious party freeze the counterparty's order and capital indefinitely at no cost to itself. The protocol exposes two natural withholding points, each backstopped by an on-chain timeout. Not every abort, however, warrants a penalty: what calls for an asymmetric penalty is the deliberate withholding of a result already computed.

The comparison phase: aborting mid-computation versus withholding a computed result. An abort in the comparison phase falls into two cases whose consequences differ sharply.

Case 1: aborting during the MPC. A malicious party may unilaterally terminate the protocol before the secure computation completes. Because the underlying MPC is maliciously secure, and because b and π_{cmp} are produced—in shared form—only upon completion, a party that aborts midway *learns nothing*: its view is simulatable, so it cannot infer the counterparty's order quantity from the messages exchanged, and it is likewise *unable to produce* a valid share of π_{cmp} or of b. Neither party then holds a submittable result, so no shares reach the chain at all. After the timeout, the chain returns both orders to the order book to re-enter matching, with no penalty imposed on either party. The aborting party gains nothing from such an abort—it has obtained no information and inflicted no additional loss on its counterparty; the only consequence is that both sides have wasted one round of matching.

Case 2: withholding a computed result. Only once the MPC has completed, and both parties hold their shares of b and π_{cmp}, does unilateral withholding become a meaningful attack: at that point the compliant party has already paid the computational cost while its order remains locked. Each of the two submission steps therefore carries its own deadline of Δ_{sub} block times ($\Delta_{\mathsf{sub}} = 10$ in our instantiation): the proof shares in the first step, and the comparison-result shares in the second, once the proof has verified. If exactly one party has submitted by the deadline of either step, the silent party is treated on chain as the delaying party: its order—and the assets that order has locked—is frozen for a penalty period (72 hours in our instantiation), while the compliant party's order is released back to the book and may immediately re-enter matching.

Withholding the plaintext at settlement. After b is published, the smaller party must send the opening (q, r) of its commitment to the larger party over the point-to-point channel. This step is off chain and hence not directly observable on chain, so it is enforced by a challenge. If the smaller party (say A) does not send the plaintext within Δ_{reveal} block times ($\Delta_{\mathsf{reveal}} = 5$), the larger party B files an on-chain enforcement request carrying a public key pk_B of its choosing, compelling A to post $\mathsf{Enc}_{\mathsf{pk}_B}(q_A, r_A)$ on chain. A must respond within Δ_{resp} block times ($\Delta_{\mathsf{resp}} = 10$); failing that, A's order—and its locked assets—is frozen for the penalty period and B's order is released for re-matching. Upon a timely response, B downloads the ciphertext and decrypts it locally to recover (q_A, r_A), then checks whether $\mathsf{Com}(q_A, r_A)$ equals A's on-chain commitment cm_{q_A}. If it does, B completes verification and settlement exactly as above. If it does not—that is, A has posted a plaintext inconsistent with its own commitment—B posts the offending plaintext (q_A, r_A) on chain and initiates adjudication. On-chain logic then checks, in order: (1) that the plaintext indeed originated from A, by re-encrypting it under B's public key pk_B and comparing against the ciphertext $\mathsf{Enc}_{\mathsf{pk}_B}(q_A, r_A)$ that A previously posted; and (2) if it did originate from A, whether $\mathsf{Com}(q_A, r_A) = \mathsf{cm}_{q_A}$ holds. If check (1) passes but check (2) fails, A is proven to have submitted a false plaintext: A's order is frozen for the penalty period and B's order is released for re-matching.

Routing the response through the chain serves two purposes. First, it makes the disclosure publicly attributable and adjudicable: either a valid response exists on chain by the deadline or it does not, and if a false plaintext appears, the two checks above let the chain assign blame unambiguously—all without the honest party ever revealing its private key. Second, on the honest path it preserves privacy: by the semantic security of the encryption scheme, the posted ciphertext reveals nothing to third parties, and its plaintext content is precisely the disclosure the larger party is already entitled to receive. (The adjudication path does expose (q_A, r_A) on chain, but only once A is proven to have cheated; at that point A has forfeited any legitimate claim to privacy, and its true quantity would in any case have been revealed to B by settlement.)

The penalty is deliberately asymmetric—the staller is frozen while the victim is freed—which makes withholding strictly unprofitable: the withholding party forfeits market access for the entire penalty period, whereas the honest party's cost is bounded by the timeout delay before its order re-enters the book.

The *form* and *duration* of the penalty are equally deliberate. We freeze the offending party's order, and the assets it has locked, for a period (typically several days) rather than slashing its funds. We do not slash because a missed response is not necessarily malicious: the counterparty may genuinely have missed the deadline due to a network fault or disconnection, and confiscating assets would be an unduly harsh response to such cases. The point of a temporary freeze is instead to make the delaying party *miss the prevailing market conditions and trading opportunities*: while its order is frozen, the released honest trader is free to complete its trades under the current market. The deterrent thus derives from *opportunity cost* rather than loss of principal—for a parasitic trader whose aim is to lock up a counterparty's capital and wait for an advantageous moment, forfeiting the market window alone renders the strategy unprofitable, while for an honest

party merely knocked offline the cost is bounded to missing one window of market activity rather than losing assets.

VII. Related Work

Prior work replaces a venue's trusted operator with secure computation. Cartlidge et al. [5] run the matching engine itself inside MPC, finding a continuous double auction impractical there while volume matching is viable. Prime Match [4] secures matching alone, returning the minimum of two committed quantities; settlement then proceeds through the bank's ordinary channels.

Invisibook keeps the premise, and as in Prime Match computes on committed inputs proved consistent in zero knowledge, but inverts the placement: matching is public, on posted sides and limit prices, and MPC is confined to settlement—a single comparison between the two counterparties, bound to on-chain commitments. Prime Match's client-to-client matching is server-aided and the bank learns every matched quantity; Invisibook runs between the counterparties alone and is publicly verifiable on chain.

Acknowledgment

The authors would like to thank...

References

[1] Foundation, Lindsay X. Lin, Legal Counsel at Interstellar and Stellar Development, "Deconstructing Decentralized Exchanges," *Stanford Journal of Blockchain Law & Policy*, jan 5 2019.

[2] L. E. Harris, "Order exposure and parasitic traders," University of Southern California, Working Paper, 1997.

[3] L. Harris, *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2002.

[4] A. Polychroniadou, G. Asharov, B. Diamond, T. Balch, H. Buehler, R. Hua, S. Gu, G. Gimler, and M. Veloso, "Prime match: a privacy-preserving inventory matching system," in *Proceedings of the 32nd USENIX Conference on Security Symposium*, ser. SEC '23, USA, 2023.

[5] J. Cartlidge, N. P. Smart, and Y. Talibi Alaoui, "Mpc joins the dark side," in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*, ser. Asia CCS '19, New York, NY, USA, 2019, p. 148–159.

[6] C. Comerton-Forde and T. J. Putniņš, "Dark trading and price discovery," *Journal of Financial Economics*, vol. 118, no. 1, pp. 70–92, 2015.

[7] H. Bessembinder, M. Panayides, and K. Venkataraman, "Hidden liquidity: An analysis of order exposure strategies in electronic stock markets," *Journal of Financial Economics*, vol. 94, no. 3, pp. 361–383, 2009.

[8] N. Hautsch and R. Huang, "On the dark side of the market: Identifying and analyzing hidden order placements," Center for Financial Studies, CFS Working Paper 2012/04, 2012.

[9] H. L. Christensen and R. Woodmansey, "Prediction of hidden liquidity in the limit order book of GLOBEX futures," *The Journal of Trading*, vol. 8, no. 3, 2013.

[10] D. Zotikov and A. Antonov, "CME iceberg order detection and prediction," *Quantitative Finance*, vol. 21, no. 11, pp. 1977–1992, 2021.

[11] A. Ozdemir and D. Boneh, "Experimenting with collaborative zk-SNARKs: Zero-Knowledge proofs for distributed secrets," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 4291–4308. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/ozdemir

[12] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Advances in Cryptology – CRYPTO 2012*, ser. Lecture Notes in Computer Science, vol. 7417. Springer, 2012, pp. 643–662.

[13] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from bitcoin," in *2014 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2014, pp. 459–474.

[14] D. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, "Zcash protocol specification," Zerocoin Electric Coin Company, Tech. Rep., 2016.

[15] J. Katz, *Digital Signatures: Background and Definitions*. Boston, MA: Springer US, 2010, pp. 3–33.

[16] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *13th USENIX Security Symposium (USENIX Security 04)*. USENIX Association, 2004. [Online]. Available: https://www.usenix.org/legacy/events/sec04/tech/dingledine.html

[17] A. Ozdemir and D. Boneh, "Experimenting with collaborative zk-SNARKs: Zero-knowledge proofs for distributed secrets," in *USENIX Security 2022*, K. R. B. Butler and K. Thomas, Eds. Boston, MA, USA: USENIX Association, Aug. 10–12, 2022, pp. 4291–4308. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/ozdemir

[18] X. Liu, Z. Zhou, Y. Wang, Y. Pang, J. He, B. Zhang, X. Yang, and J. Zhang, "Scalable collaborative zk-SNARK and its application to fully distributed proof delegation," in *USENIX Security 2025*, L. Bauer and G. Pellegrino, Eds. Seattle, WA, USA: USENIX Association, Aug. 13–15, 2025, pp. 2025–2044. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/liu-xuanming

[19] S. Garg, A. Goel, A. Jain, B. Roberts, and S. Sekar, "Malicious security in collaborative zk-SNARKs: More than meets the eye," in *CRYPTO 2025, Part VII*, ser. LNCS, Y. T. Kalai and S. F. Kamara, Eds., vol. 16006. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 17–21, 2025, pp. 396–428.